Java Script

1 Introduction to JavaScript

1.1 What is JavaScript?

JavaScript (JS) is a lightweight, interpreted, high-level programming language that runs inside the web browser (and also on servers through Node.js). It is the language that makes a web page *interactive* and *alive*.

Think of a website like a human body:

- **HTML** is the *skeleton* — it gives structure (headings, paragraphs, buttons).
- **CSS** is the *skin and clothes* — it gives colour, style and design.
- **JavaScript** is the *brain and muscles* — it makes things move, react and think.

> **Key Point**
>
> HTML = Structure, CSS = Presentation, JavaScript = Behaviour.

1.2 What can JavaScript do?

- React to user actions (click, hover, typing, scrolling).
- Change the content and style of a page without reloading it.
- Validate forms before sending data to the server.
- Create animations, sliders, pop-ups, games and calculators.
- Fetch data from a server (AJAX / Fetch API) and display it.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Store data in the browser using `localStorage`.

1.3 How to add JavaScript to an HTML page

There are three ways to include JavaScript in an HTML document:

```HTML — THREE WAYS TO INCLUDE JS
<!-- 1. INLINE JavaScript (written directly inside an HTML attribute
<button onclick="alert('Hello!')">Click Me</button>

<!-- 2. INTERNAL JavaScript (written inside a <script> tag) -->
<script>
  console.log("Hello from the internal script");
</script>

<!-- 3. EXTERNAL JavaScript (written in a separate .js file) -->
<script src="script.js"></script>
```

Method	Where it is written	Best used for
Inline	Inside HTML attributes like `onclick`	Very tiny, quick tests only
Internal	Inside `<script>` in the same HTML file	Small pages / single-file demos
External	In a separate `.js` file linked with `src`	**Recommended** — clean and reusable

1.4 Your first JavaScript program

```JAVASCRIPT
console.log("Hello, World!");
```

```
▸ OUTPUT
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Hello, World!
```

`console.log()` prints a message in the browser's Developer Console (press **F12** → Console tab). It is the most common way to check what your code is doing.

1.5 Basic syntax rules

- **Case sensitive:** `name`, `Name` and `NAME` are three different things.
- **Statements:** each instruction is a statement. Statements usually end with a semicolon `;` (optional but recommended).
- **Comments:** used to explain code. They are ignored by the browser.
- **Whitespace:** spaces and new lines do not matter to JavaScript (but they matter to humans!).

JAVASCRIPT – COMMENTS

```
// This is a single-line comment

/* This is a
   multi-line comment
   spanning several lines */

let x = 10;  // statement ends with a semicolon
```

1.6 Output methods

Method	What it does
`console.log()`	Prints to the browser console (used by developers)
`alert()`	Shows a pop-up dialog box to the user
`document.write()`	Writes directly into the HTML page
`prompt()`	Takes input from the user through a pop-up

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

> **Tip**
>
> Always keep the **Console** open (F12) while learning JavaScript. It shows your output and also points out your errors with the exact line number.

2 Variables

2.1 What is a variable?

A **variable** is a named container (a box) that stores a value in memory. Once you store a value in a variable, you can use it again and again, change it, or pass it around.

> **Simple analogy**
>
> A variable is like a labelled box. The label is the *variable name*, and the thing inside the box is the *value*.

2.2 Declaring a variable

In modern JavaScript there are three keywords to create a variable: `var`, `let`, and `const`.

JAVASCRIPT — DECLARING VARIABLES

```
let name = "Riya";      // can be changed later
const PI = 3.14159;     // cannot be re-assigned
var city = "Delhi";     // old style, avoid in modern code

console.log(name);      // Riya
console.log(PI);        // 3.14159
console.log(city);      // Delhi
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

2.3 var vs let vs const

Feature	`var`	`let`	`const`
Can be re-assigned	Yes	Yes	No
Must be given a value at declaration	No	No	Yes
Scope	Function scope	Block scope `{ }`	Block scope `{ }`
Can be re-declared	Yes	No	No
Recommended today?	No (legacy)	Yes	Yes (default choice)

> **Rule of thumb**
>
> Use `const` by default. Use `let` only when the value needs to change. Avoid `var` in new code.

2.4 Rules for naming variables

- Names can contain letters, digits, `_` and `$`.
- Names **cannot** start with a digit. `let 1name = "x";` ❌
- Names are case sensitive: `age` ≠ `Age`.
- Reserved words cannot be used: `let`, `if`, `for`, `function`, etc.
- No spaces allowed. Use **camelCase**: `firstName`, `totalPrice`.
- Use meaningful names — `userAge` is better than `x`.

JAVASCRIPT — NAMING EXAMPLES

```
let firstName = "Amit";      // ✅ camelCase (recommended)
let _count = 5;              // ✅ starts with underscore
let $price = 99;             // ✅ starts with dollar sign
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// let 2fast = 10;          ❌ cannot start with a digit
// let first name = "A";    ❌ no spaces allowed
// let for = 1;             ❌ reserved keyword
```

2.5 Data types

JavaScript variables can hold different types of values. The main ones are:

Type	Description	Example
String	Text inside quotes	`"Hello"`, `'JS'`
Number	Integers and decimals	`25`, `3.14`
Boolean	True or false only	`true`, `false`
Undefined	Declared but no value assigned	`let x;`
Null	Intentionally empty value	`let y = null;`
Object	Collection of key–value pairs	`{name: "Riya", age: 20}`
Array	List of values (a type of object)	`[10, 20, 30]`
BigInt	Very large integers	`12345678901234567890n`
Symbol	Unique identifier	`Symbol("id")`

JAVASCRIPT — DATA TYPES IN ACTION

```
let fullName = "Priya Sharma";   // String
let age      = 21;               // Number
let isStudent = true;            // Boolean
let address;                     // Undefined
let middleName = null;           // Null
let marks = [85, 90, 78];        // Array
let person = { name: "Priya", age: 21 };  // Object

console.log(typeof fullName);   // string
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(typeof age);         // number
console.log(typeof isStudent);   // boolean
console.log(typeof address);     // undefined
console.log(typeof middleName); // object  (a famous JavaScript quir
console.log(typeof marks);       // object
console.log(typeof person);      // object
```

> **Note**
>
> JavaScript is a **dynamically typed** language. You do not have to declare the type — the type is decided automatically from the value. A variable can even change its type later (though it is better not to do that).

2.6 Re-assigning values

JAVASCRIPT

```
let score = 10;
score = 25;                 // ✅ allowed with let
console.log(score);         // 25

const total = 100;
// total = 200;             ❌ TypeError: Assignment to constant var
```

> **Important**
>
> `const` means the *variable binding* cannot change — not that the contents are frozen. You can still modify an object or array declared with `const`:

JAVASCRIPT — CONST WITH OBJECTS AND ARRAYS

```
const user = { name: "Riya" };
user.name = "Priya";        // ✅ allowed (content changed, not the
console.log(user.name);     // Priya

const numbers = [1, 2, 3];
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
numbers.push(4);            // ✅ allowed
console.log(numbers);       // [1, 2, 3, 4]
```

2.7 Scope in one minute

- **Global scope:** declared outside any block — available everywhere.
- **Function scope:** declared inside a function — available only inside it.
- **Block scope:** declared inside `{ }` with `let` / `const` — available only inside that block.

```
JAVASCRIPT — BLOCK SCOPE

{
  let blockVar = "inside";
  console.log(blockVar);   // inside
}
// console.log(blockVar);  ❌ ReferenceError: blockVar is not define
```

2.8 Common variable mistakes

- Using a variable before declaring it.
- Forgetting quotes around strings: `let name = Riya;` ❌
- Re-declaring with `let` in the same scope.
- Using `const` without an initial value.
- Using confusing names like `a`, `b`, `temp1` everywhere.

3 JavaScript Operators

3.1 What is an operator?

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

An **operator** is a symbol that performs an operation on values (called *operands*). For example, in `10 + 5`, the `+` is the operator and `10` and `5` are the operands.

3.2 Arithmetic operators

Operator	Name	Example	Result
`+`	Addition	`10 + 3`	13
`-`	Subtraction	`10 - 3`	7
`*`	Multiplication	`10 * 3`	30
`/`	Division	`10 / 4`	2.5
`%`	Modulus (remainder)	`10 % 3`	1
`**`	Exponent (power)	`2 ** 3`	8

JAVASCRIPT — ARITHMETIC

```
let a = 10;
let b = 3;

console.log(a + b);   // 13
console.log(a - b);   // 7
console.log(a * b);   // 30
console.log(a / b);   // 3.3333333333333335
console.log(a % b);   // 1   (remainder of 10 ÷ 3)
console.log(a ** b);  // 1000 (10 to the power 3)
```

Useful trick

The modulus operator `%` is great for checking even/odd: `num % 2 === 0` means the number is even.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3.3 Assignment operators

Assignment operators store a value into a variable, often combining an operation at the same time.

Operator	Example	Same as
`=`	`x = 5`	Assign 5 to x
`+=`	`x += 5`	`x = x + 5`
`-=`	`x -= 5`	`x = x - 5`
`*=`	`x *= 5`	`x = x * 5`
`/=`	`x /= 5`	`x = x / 5`
`%=`	`x %= 5`	`x = x % 5`
`**=`	`x **= 2`	`x = x ** 2`

JAVASCRIPT — ASSIGNMENT

```
let x = 10;
x += 5;   console.log(x);   // 15
x -= 3;   console.log(x);   // 12
x *= 2;   console.log(x);   // 24
x /= 4;   console.log(x);   // 6
x %= 4;   console.log(x);   // 2
```

3.4 Comparison operators

Comparison operators compare two values and always return a **Boolean** (`true` or `false`).

Operator	Meaning	Example	Result
`==`	Equal to (value only)	`5 == "5"`	`true`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Operator	Meaning	Example	Result
`===`	Strictly equal (value + type)	`5 === "5"`	`false`
`!=`	Not equal (value only)	`5 != "5"`	`false`
`!==`	Strictly not equal	`5 !== "5"`	`true`
`>`	Greater than	`10 > 5`	`true`
`<`	Less than	`10 < 5`	`false`
`>=`	Greater than or equal to	`10 >= 10`	`true`
`<=`	Less than or equal to	`10 <= 9`	`false`

3.5 == vs === (very important)

JAVASCRIPT — == VS ===

```
console.log(5 == "5");     // true   → only values compared, "5" conv
console.log(5 === "5");    // false  → values AND types compared (num

console.log(0 == false);  // true
console.log(0 === false); // false

console.log(null == undefined);   // true
console.log(null === undefined);  // false
```

Best practice

Always prefer `===` and `!==`. The loose `==` performs automatic type conversion, which causes confusing bugs.

3.6 Logical operators

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Operator	Name	Returns true when…
`&&`	AND	**both** conditions are true
`||`	OR	**at least one** condition is true
`!`	NOT	the condition is false (it reverses the result)

JAVASCRIPT — LOGICAL OPERATORS

```
let age = 20;
let hasID = true;

console.log(age >= 18 && hasID);   // true  (both true)
console.log(age >= 18 || hasID);   // true  (at least one true)
console.log(!hasID);               // false (reversed)

// Truth table for AND
console.log(true  && true);   // true
console.log(true  && false);  // false
console.log(false && true);   // false
console.log(false && false);  // false

// Truth table for OR
console.log(true  || false);  // true
console.log(false || false);  // false
```

3.7 Unary operators & increment / decrement

Operator	Name	Example	Result
`++`	Increment (add 1)	`x++`	x increases by 1
`--`	Decrement (subtract 1)	`x--`	x decreases by 1
`-`	Unary minus (negate)	`-x`	opposite sign
`!`	Logical NOT	`!true`	`false`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Operator	Name	Example	Result
`typeof`	Type of a value	`typeof 5`	`"number"`

JAVASCRIPT — PRE VS POST INCREMENT

```
let a = 5;

console.log(a++);   // 5   → prints first, THEN increases (post-incr
console.log(a);     // 6

let b = 5;
console.log(++b);   // 6   → increases first, THEN prints (pre-incre
console.log(b);     // 6
```

3.8 String operators (concatenation)

The `+` operator joins (concatenates) strings together.

JAVASCRIPT — STRING CONCATENATION

```
let first = "Hello";
let second = "World";

console.log(first + " " + second);   // Hello World
console.log("Age: " + 20);           // Age: 20  (number converted t

// Careful! This is a common trap
console.log("5" + 3);   // "53"   (string concatenation)
console.log("5" - 3);   // 2      (number subtraction)
console.log(5 + 3 + "2"); // "82" (5+3=8 first, then joined with "2"
```

Modern alternative — Template literals

Use backticks `` ` ` `` and `${ }` to insert variables into strings cleanly.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — TEMPLATE LITERALS

```
let name = "Riya";
let age = 20;

console.log(`My name is ${name} and I am ${age} years old.`);
// My name is Riya and I am 20 years old.

console.log(`2 + 3 = ${2 + 3}`);   // 2 + 3 = 5
```

3.9 Ternary (conditional) operator

A short way to write an `if...else` in one line.

Syntax: `condition ? valueIfTrue : valueIfFalse`

JAVASCRIPT — TERNARY

```
let age = 20;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result);   // Adult

// Same thing using if...else
let result2;
if (age >= 18) {
  result2 = "Adult";
} else {
  result2 = "Minor";
}
```

3.10 Operator precedence (order of evaluation)

When an expression has several operators, JavaScript follows a fixed priority order:

Priority	Operators
1 (highest)	`( )` grouping
2	`!`, `++`, `--`, `typeof`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Priority	Operators
3	`**`
4	`*`, `/`, `%`
5	`+`, `-`
6	`<`, `<=`, `>`, `>=`
7	`==`, `===`, `!=`, `!==`
8	`&&`
9	`||`
10 (lowest)	`=`, `+=`, `-=`, ...

JAVASCRIPT — PRECEDENCE EXAMPLE

```
console.log(2 + 3 * 4);      // 14  (multiplication first)
console.log((2 + 3) * 4);    // 20  (brackets first)

console.log(10 - 2 + 3);     // 11  (left to right for same priority

console.log(5 > 3 && 2 < 4);  // true
```

Remember

When in doubt, use parentheses `( )`. They make your intention clear and remove any confusion about the order of evaluation.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4 Conditional Statements

4.1 What is a conditional statement?

A **conditional statement** lets your program make decisions. It runs a block of code only if a given condition is `true`, and optionally runs different code if it is `false`.

> **Analogy**
>
> "If it is raining, take an umbrella; otherwise, wear sunglasses." That's exactly what an `if...else` statement does in code.

4.2 The if statement

Syntax:

JAVASCRIPT — IF

```
if (condition) {
  // this block runs only when the condition is true
}
```

JAVASCRIPT — IF EXAMPLE

```
let temperature = 35;

if (temperature > 30) {
  console.log("It is very hot today!");
}
// Output: It is very hot today!
```

4.3 The if...else statement

Runs one block when the condition is true, and another block when it is false.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — IF...ELSE

```
let age = 16;

if (age >= 18) {
  console.log("You can vote.");
} else {
  console.log("You are too young to vote.");
}
// Output: You are too young to vote.
```

4.4 The else if ladder

Used when there are more than two possible outcomes. JavaScript checks each condition from top to bottom and stops at the **first true** one.

JAVASCRIPT — ELSE IF

```
let marks = 78;

if (marks >= 90) {
  console.log("Grade A+");
} else if (marks >= 80) {
  console.log("Grade A");
} else if (marks >= 70) {
  console.log("Grade B");
} else if (marks >= 60) {
  console.log("Grade C");
} else {
  console.log("Grade F");
}
// Output: Grade B
```

Order matters

Always write the most restrictive condition first. If you write `marks >= 60` at the top, every student scoring 60 or more would get "Grade C" and the later conditions would never be checked.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4.5 Nested if statements

An `if` statement written inside another `if` statement.

JAVASCRIPT — NESTED IF

```
let hasLicense = true;
let age = 22;

if (age >= 18) {
  if (hasLicense) {
    console.log("You can drive.");
  } else {
    console.log("Get a license first.");
  }
} else {
  console.log("You are underage.");
}
// Output: You can drive.
```

4.6 The switch statement

`switch` is a cleaner alternative to a long `else if` ladder when you are comparing **one variable** against many fixed values.

JAVASCRIPT — SWITCH

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  case 4:
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    console.log("Thursday");
    break;
  case 5:
    console.log("Friday");
    break;
  default:
    console.log("Weekend or invalid day");
}
// Output: Wednesday
```

Do not forget break

Without `break`, execution "falls through" to the next case and keeps running. This is sometimes useful, but usually it is a bug.

JAVASCRIPT — FALL-THROUGH EXAMPLE

```
let fruit = "apple";

switch (fruit) {
  case "apple":
  case "mango":
  case "banana":
    console.log("This is a fruit.");
    break;                  // all three cases share this code
  default:
    console.log("Unknown item.");
}
// Output: This is a fruit.
```

4.7 Truthy and Falsy values

In a condition, JavaScript converts the value to a Boolean. Some values are automatically treated as **false** (falsy); everything else is **true** (truthy).

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Falsy values (treated as false)	Truthy values (treated as true)
`false`	`true`
`0`	Any non-zero number (`1` , `-5` , `3.14`)
`""` (empty string)	Any non-empty string (`"hi"` , `"0"`)
`null`	Objects and arrays (`{}` , `[]`)
`undefined`	Functions
`NaN`	

JAVASCRIPT — TRUTHY / FALSY

```
if ("") {
  console.log("This will NOT print");
}

if ("hello") {
  console.log("This WILL print");
}

let name = "";
if (name) {
  console.log("Hello, " + name);
} else {
  console.log("Name is empty!");   // this runs
}
```

4.8 Common conditional mistakes

- **Using `=` instead of `==`:** `if (x = 5)` assigns 5 and is always true. Use `if (x === 5)`.
- **Forgetting curly braces:** without `{ }`, only the first statement belongs to the `if`.
- **Wrong condition order** in an `else if` ladder.
- **Missing `break`** in a `switch`.
- **Using `==`** where `===` is safer.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5 JavaScript Loops

5.1 Why do we need loops?

A **loop** repeats a block of code many times without writing it again and again. If you want to print the numbers 1 to 100, you would not write 100 `console.log` statements — you would write a loop.

> **Definition**
>
> A loop continues repeating as long as its condition remains `true`.

JavaScript has the following main loop types:

- `for` loop — when you know how many times to repeat.
- `while` loop — when you repeat as long as a condition is true.
- `do...while` loop — like `while`, but runs at least once.
- `for...of` — to loop through the *values* of an array/string.
- `for...in` — to loop through the *keys* of an object.

5.2 The for loop

Syntax:

```
JAVASCRIPT — FOR LOOP SYNTAX

for (initialization; condition; increment) {
  // code to repeat
}
```

The three parts are:

1. **Initialization** — runs once before the loop starts (e.g. `let i = 1`).
2. **Condition** — checked before every iteration; the loop runs while it is true.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3. **Increment / Decrement** — runs after every iteration (e.g. `i++`).

JAVASCRIPT — FOR LOOP EXAMPLE

```javascript
for (let i = 1; i <= 5; i++) {
  console.log("Number: " + i);
}
```

▸ OUTPUT

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

COUNTING BACKWARDS

JAVASCRIPT

```javascript
for (let i = 5; i >= 1; i--) {
  console.log(i);
}
// 5 4 3 2 1
```

LOOPING THROUGH AN ARRAY

JAVASCRIPT

```javascript
let fruits = ["apple", "banana", "mango"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
// apple
// banana
// mango
```

SUM OF NUMBERS USING A LOOP

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
  sum += i;
}

console.log("Sum = " + sum);    // Sum = 55
```

5.3 The while loop

A `while` loop checks the condition *before* each iteration. If the condition is false at the beginning, the loop body never runs.

JAVASCRIPT — WHILE LOOP

```
let i = 1;

while (i <= 5) {
  console.log("i = " + i);
  i++;              // IMPORTANT: must change i, otherwise infinite
}
```

▸ OUTPUT

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

Infinite loop danger

If you forget to update the variable inside a `while` loop, the condition stays true forever and your browser freezes.

5.4 The do...while loop

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

A `do...while` loop runs the body **first** and checks the condition **afterwards**. So the body always executes at least once, even if the condition is false.

JAVASCRIPT — DO...WHILE

```
let i = 1;

do {
  console.log("i = " + i);
  i++;
} while (i <= 5);
```

▸ OUTPUT

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

PROOF THAT DO...WHILE RUNS AT LEAST ONCE

JAVASCRIPT

```
let n = 100;

do {
  console.log("This prints once even though n > 10 is false");
  n++;
} while (n < 10);
```

5.5 The for...of loop

Used to iterate over the **values** of an iterable (array, string, set, map).

JAVASCRIPT — FOR...OF

```
let colors = ["red", "green", "blue"];

for (let color of colors) {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  console.log(color);
}
// red
// green
// blue

let word = "Hi";
for (let letter of word) {
  console.log(letter);
}
// H
// i
```

5.6 The for...in loop

Used to iterate over the **keys (property names)** of an object.

JAVASCRIPT — FOR...IN

```
let student = { name: "Riya", age: 20, course: "BCA" };

for (let key in student) {
  console.log(key + " = " + student[key]);
}
// name = Riya
// age = 20
// course = BCA
```

> **for...of vs for...in**
>
> Use `for...of` for **arrays** (gives values) and `for...in` for **objects** (gives keys).

5.7 Nested loops

A loop inside another loop. The inner loop completes fully for every single iteration of the outer loop.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — NESTED LOOPS (MULTIPLICATION TABLE)

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    console.log(i + " x " + j + " = " + (i * j));
  }
}
```

▸ OUTPUT

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
```

STAR PATTERN USING NESTED LOOPS

JAVASCRIPT

```
for (let i = 1; i <= 5; i++) {
  let row = "";
  for (let j = 1; j <= i; j++) {
    row += "* ";
  }
  console.log(row);
}
```

▸ OUTPUT

```
*
* *
* * *
* * * *
* * * * *
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5.8 Comparison of loop types

Loop	Condition checked	Minimum runs	Best for
`for`	Before each iteration	0	Known number of repetitions
`while`	Before each iteration	0	Unknown count, condition-based
`do...while`	After each iteration	1	Must run at least once
`for...of`	Automatic	0	Values of arrays/strings
`for...in`	Automatic	0	Keys of objects

5.9 Infinite loops

JAVASCRIPT — INFINITE LOOP (DO NOT RUN)

```
// while (true) {
//   console.log("This never stops!");
// }

// for (let i = 1; i > 0; i++) {
//   console.log(i);      // i always greater than 0 → never ends
// }
```

Warning

An infinite loop will freeze the browser tab. If it happens, close the tab. Always make sure the loop variable eventually makes the condition false.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

6 Break and Continue Statements

6.1 What are they?

`break` and `continue` are **loop control statements**. They let you change the normal flow of a loop from inside the loop body.

Statement	Effect
`break`	**Stops the loop completely** and jumps to the code after the loop.
`continue`	**Skips the current iteration** and jumps to the next one.

6.2 The break statement

Use `break` when you want to stop looping as soon as something is found.

JAVASCRIPT — BREAK IN A FOR LOOP

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    break;              // stop the loop when i reaches 5
  }
  console.log(i);
}
```

▸ OUTPUT

```
1
2
3
4
```

The loop stops at `i = 5`. Numbers 5 to 10 are never printed, and the increment never continues past that point.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

PRACTICAL EXAMPLE — SEARCH FOR A VALUE

JAVASCRIPT

```
let numbers = [3, 8, 12, 7, 20, 15];
let target = 7;
let found = false;

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] === target) {
    console.log("Found " + target + " at index " + i);
    found = true;
    break;                // no need to keep searching
  }
}

if (!found) {
  console.log("Not found");
}
```

▸ OUTPUT

```
Found 7 at index 3
```

6.3 The continue statement

Use `continue` to skip the rest of the current iteration and move to the next one.

JAVASCRIPT — CONTINUE IN A FOR LOOP

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue;             // skip even numbers
  }
  console.log(i);
}
```

▸ OUTPUT

```
1
3
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
5
7
9
```

When `i` is even, `continue` skips the `console.log` and jumps to the next iteration — so only odd numbers are printed.

ANOTHER EXAMPLE — SKIPPING A SPECIFIC VALUE

JAVASCRIPT

```
let fruits = ["apple", "banana", "mango", "banana", "kiwi"];

for (let fruit of fruits) {
  if (fruit === "banana") {
    continue;            // skip bananas
  }
  console.log(fruit);
}
// apple
// mango
// kiwi
```

6.4 break vs continue — side by side

Loop with `break`

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) break;
  console.log(i);
}
// 1
// 2
```

Loop with `continue`

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);
}
// 1
// 2
// 4
// 5
```

6.5 break and continue in while and do...while

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — WHILE WITH BREAK

```
let num = 1;

while (true) {           // intentionally infinite
  if (num > 5) {
    break;               // exit condition
  }
  console.log(num);
  num++;
}
// 1 2 3 4 5
```

JAVASCRIPT — DO...WHILE WITH CONTINUE

```
let i = 0;

do {
  i++;
  if (i === 3) {
    continue;            // skip printing 3
  }
  console.log(i);
} while (i < 5);
// 1
// 2
// 4
// 5
```

Careful with continue in a while loop

If you use `continue` in a `while` loop before the increment statement, the variable may never change and you will get an infinite loop.

JAVASCRIPT — BAD VS GOOD

```
// ❌ BAD — infinite loop
// let i = 0;
// while (i < 5) {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
//   if (i === 2) continue;   // i never increments!
//   console.log(i);
//   i++;
// }

// ✅ GOOD — increment before continue
let j = 0;
while (j < 5) {
  j++;
  if (j === 2) continue;
  console.log(j);
}
// 1 3 4 5
```

6.6 break inside switch vs inside loop

Inside a `switch` statement, `break` only exits the switch. Inside a loop, it exits the loop. If a `switch` is inside a loop, `break` exits only the switch, not the loop.

JAVASCRIPT — SWITCH INSIDE A LOOP

```
for (let i = 1; i <= 3; i++) {
  switch (i) {
    case 1:
      console.log("One");
      break;            // exits the switch only
    case 2:
      console.log("Two");
      break;
    default:
      console.log("Other");
  }
}
// One
// Two
// Other
```

6.7 Labeled break and continue (advanced)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In nested loops, labels let you break out of an outer loop.

JAVASCRIPT — LABELED BREAK

```
outer: for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (i === 2 && j === 2) {
      break outer;       // exits BOTH loops
    }
    console.log(i + ", " + j);
  }
}
// 1, 1
// 1, 2
// 1, 3
// 2, 1
```

> **Note**
>
> Labels are rarely used in everyday code. They are useful to know, but prefer clear, simple loops when possible.

7 Quick Reference Summary

Topic	Key points to remember
JavaScript	Makes web pages interactive. Added via inline, internal or external script.
Variables	`let` (changeable), `const` (fixed binding), `var` (old). Use camelCase and meaningful names.
Data types	String, Number, Boolean, Undefined, Null, Object, Array, BigInt,

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Topic	Key points to remember
	Symbol.
Arithmetic	`+ - * / % **`
Comparison	`== === != !== > < >= <=` — prefer `===`.
Logical	`&&` (AND), `||` (OR), `!` (NOT).
Conditionals	`if`, `if...else`, `else if`, `switch`, ternary.
Loops	`for`, `while`, `do...while`, `for...of`, `for...in`.
break	Exits the loop immediately.
continue	Skips the current iteration only.

8 Practice Exercises

Try these in the browser console. They use only the concepts covered above.

1. Create three variables: your name (const), your age (let), and your city (let). Print a sentence using a template literal.

2. Take two numbers and print their sum, difference, product, quotient and remainder.

3. Write a program that checks whether a number is even or odd using `%`.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4. Write an `if...else if...else` program that prints a grade based on marks (A+, A, B, C, F).

5. Rewrite exercise 4 using a `switch` statement.

6. Print all numbers from 1 to 50 using a `for` loop.

7. Print the multiplication table of 7 using a `for` loop.

8. Find the sum of all numbers from 1 to 100 using a loop.

9. Print only the odd numbers from 1 to 20 using `continue`.

10. Search for the number 25 in an array and stop the loop using `break` when it is found.

11. Print a triangle star pattern of 5 rows using nested loops.

12. Loop through an object `{ name: "Riya", age: 20, city: "Pune" }` using `for...in` and print each key with its value.

Final advice

Programming is learned by **doing**, not by reading. Type every example yourself, change the values, break the code on purpose, and see what error the console gives you. That is how the concepts truly stick.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1 Dialog Boxes

1.1 What is a dialog box?

A **dialog box** is a small pop-up window that the browser shows on top of the page to *inform* the user, *ask* the user something, or *confirm* an action. JavaScript gives us three built-in dialog boxes: `alert()`, `confirm()`, and `prompt()`.

All three are methods of the global `window` object, so `window.alert()` and `alert()` mean exactly the same thing.

> **Important behaviour**
>
> Dialog boxes are **blocking**. While a dialog is open, the rest of the page freezes and the user cannot click anything else. Use them sparingly — mostly for learning and simple testing.

1.2 The three dialog boxes at a glance

Method	Purpose	Buttons shown	Returns
`alert()`	Shows a message to the user	OK only	`undefined`
`confirm()`	Asks the user to confirm something	OK and Cancel	`true` (OK) or `false` (Cancel)
`prompt()`	Asks the user to type a value	OK and Cancel + text field	The typed text (string), or `null` if cancelled

1.3 alert()

Shows a simple message with an OK button. It is used only to *display* information.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT – ALERT()

```
alert("Welcome to my website!");

// You can combine text and variables
let name = "Riya";
alert("Hello, " + name + "!");

// Better: template literals
alert(`Hello, ${name}!`);
```

> **Note**
>
> `alert()` converts whatever you pass into a string. Numbers and booleans are printed as text.

1.4 confirm()

Shows a message with **OK** and **Cancel** buttons. It returns a Boolean, so it is normally used inside an `if` statement.

JAVASCRIPT – CONFIRM()

```
let answer = confirm("Do you really want to delete this record?");

if (answer) {
  console.log("Record deleted.");
} else {
  console.log("Action cancelled.");
}
```

JAVASCRIPT – CONFIRM BEFORE LEAVING A PAGE

```
let leave = confirm("You have unsaved changes. Leave anyway?");

if (leave) {
  alert("You left the page.");
} else {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  alert("You stayed. Good choice!");
}
```

1.5 prompt()

Shows a text box where the user can type something, along with OK and Cancel buttons. The second argument (optional) is the *default value* shown inside the box.

JAVASCRIPT — PROMPT()

```
let userName = prompt("What is your name?");

if (userName !== null) {
  alert("Hello, " + userName + "!");
} else {
  alert("You cancelled the prompt.");
}
```

prompt() always returns a string

Even if the user types `25`, you receive the string `"25"`. Convert it with `Number()` or `parseInt()` before doing maths.

JAVASCRIPT — CONVERTING PROMPT INPUT

```
let input = prompt("Enter a number:");

// ❌ Wrong: this joins strings
// let doubled = input * 2;      // works for * but not +

// ✅ Correct: convert to a number first
let num = Number(input);

if (isNaN(num)) {
  alert("That was not a valid number!");
} else {
  alert("Double of your number is " + (num * 2));
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

A SMALL CALCULATOR USING ALL THREE

JAVASCRIPT — COMPLETE EXAMPLE

```
let a = Number(prompt("Enter first number:", "0"));
let b = Number(prompt("Enter second number:", "0"));

let result = a + b;

alert("The sum is " + result);

let again = confirm("Do you want to calculate again?");
if (again) {
  alert("Great! Refresh to start over.");
} else {
  alert("Goodbye!");
}
```

1.6 Summary of the three dialogs

Use it when…	Dialog
You only need to show a warning / message	`alert()`
You need a yes/no decision	`confirm()`
You need a value typed by the user	`prompt()`

2 JavaScript Arrays

2.1 What is an array?

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

An **array** is a single variable that can hold *many values* in an ordered list. Instead of creating `fruit1`, `fruit2`, `fruit3`, you create one array called `fruits`.

> **Analogy**
>
> An array is like a row of numbered lockers. Each locker has a number (the *index*) and contains one item (the *value*).

2.2 Creating an array

JAVASCRIPT — CREATING ARRAYS

```
// 1. Array literal (most common and recommended)
let fruits = ["apple", "banana", "mango"];

// 2. Using the Array constructor
let numbers = new Array(10, 20, 30);

// 3. Empty array, filled later
let marks = [];
marks[0] = 85;
marks[1] = 90;

// 4. Mixed data types (allowed, but usually not recommended)
let mixed = ["Riya", 21, true, null];

// 5. Array of arrays (2D array)
let grid = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(fruits);    // ["apple", "banana", "mango"]
console.log(numbers);   // [10, 20, 30]
```

2.3 Indexing and length

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Array positions start at **0**. The first element is at index 0, the second at index 1, and so on. The `length` property tells you how many elements exist.

JAVASCRIPT — ACCESSING ELEMENTS

```
let fruits = ["apple", "banana", "mango"];

console.log(fruits[0]);        // apple   (first)
console.log(fruits[1]);        // banana
console.log(fruits[2]);        // mango   (last)
console.log(fruits[3]);        // undefined (does not exist)

console.log(fruits.length);    // 3

// Last element — a handy trick
console.log(fruits[fruits.length - 1]);   // mango
```

Common mistake

Do not confuse `length` with the last index. An array of length 3 has indexes 0, 1, 2 — the last index is always `length - 1`.

2.4 Modifying array elements

JAVASCRIPT — CHANGING VALUES

```
let fruits = ["apple", "banana", "mango"];

fruits[1] = "orange";          // replace "banana"
console.log(fruits);           // ["apple", "orange", "mango"]

fruits[3] = "kiwi";            // add at the end (index 3)
console.log(fruits);           // ["apple", "orange", "mango", "kiwi

fruits.length = 2;             // truncate the array
console.log(fruits);           // ["apple", "orange"]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

2.5 Looping through an array

JAVASCRIPT — LOOPING ARRAYS

```
let colors = ["red", "green", "blue"];

// 1. Classic for loop (gives you the index)
for (let i = 0; i < colors.length; i++) {
  console.log(i + ": " + colors[i]);
}

// 2. for...of (gives you the value) — cleanest for reading
for (let color of colors) {
  console.log(color);
}

// 3. forEach() with a callback function
colors.forEach(function (color, index) {
  console.log(index, color);
});

// 4. Arrow function version
colors.forEach((color) => console.log(color));
```

▸ OUTPUT

```
0: red
1: green
2: blue
red
green
blue
```

2.6 The most important array methods

ADDING AND REMOVING ELEMENTS

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method	What it does	Changes original?	Returns
`push(x)`	Adds to the **end**	Yes	New length
`pop()`	Removes from the **end**	Yes	Removed element
`unshift(x)`	Adds to the **start**	Yes	New length
`shift()`	Removes from the **start**	Yes	Removed element
`splice(start, count, …items)`	Add / remove anywhere	Yes	Removed elements
`slice(start, end)`	Copies a portion	**No**	New array

JAVASCRIPT — PUSH / POP / SHIFT / UNSHIFT

```
let queue = ["A", "B", "C"];

queue.push("D");
console.log(queue);         // ["A", "B", "C", "D"]

queue.pop();
console.log(queue);         // ["A", "B", "C"]

queue.unshift("Z");
console.log(queue);         // ["Z", "A", "B", "C"]

queue.shift();
console.log(queue);         // ["A", "B", "C"]
```

JAVASCRIPT — SPLICE() AND SLICE()

```
let items = ["a", "b", "c", "d", "e"];
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// splice(startIndex, deleteCount, ...itemsToAdd)
items.splice(1, 2);            // removes "b" and "c"
console.log(items);            // ["a", "d", "e"]

items.splice(1, 0, "x", "y");  // inserts at index 1, deletes nothin
console.log(items);            // ["a", "x", "y", "d", "e"]

// slice(start, end) — end is NOT included, original unchanged
let part = items.slice(1, 3);
console.log(part);             // ["x", "y"]
console.log(items);            // ["a", "x", "y", "d", "e"]
```

SEARCHING

Method	Returns
`indexOf(value)`	First index, or `-1` if not found
`lastIndexOf(value)`	Last index, or `-1`
`includes(value)`	`true` / `false`
`find(fn)`	First matching element
`findIndex(fn)`	Index of first match, or `-1`

JAVASCRIPT — SEARCHING ARRAYS

```
let nums = [10, 20, 30, 20, 40];

console.log(nums.indexOf(20));       // 1
console.log(nums.lastIndexOf(20));    // 3
console.log(nums.indexOf(99));        // -1
console.log(nums.includes(30));       // true

let people = [
  { name: "Riya", age: 20 },
  { name: "Amit", age: 25 }
];
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
let found = people.find(p => p.age > 22);
console.log(found.name);              // Amit

console.log(people.findIndex(p => p.name === "Riya"));   // 0
```

TRANSFORMING ARRAYS

Method	Purpose	Returns
`map(fn)`	Transforms every element	New array of same length
`filter(fn)`	Keeps elements that pass a test	New (shorter) array
`reduce(fn, initial)`	Boils the array down to one value	Single value
`some(fn)`	Is *at least one* element true?	Boolean
`every(fn)`	Are *all* elements true?	Boolean

JAVASCRIPT — MAP / FILTER / REDUCE

```
let prices = [100, 250, 400, 50];

// map: apply a change to every element
let withTax = prices.map(p => p + p * 0.18);
console.log(withTax);          // [118, 295, 472, 59]

// filter: keep only what matches
let expensive = prices.filter(p => p > 150);
console.log(expensive);        // [250, 400]

// reduce: combine everything into one value
let total = prices.reduce((sum, p) => sum + p, 0);
console.log(total);            // 800

// some / every
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(prices.some(p => p > 300));   // true
console.log(prices.every(p => p > 100));  // false
```

SORTING, REVERSING AND JOINING

JAVASCRIPT — SORT / REVERSE / JOIN / CONCAT

```
let letters = ["c", "a", "b"];
letters.sort();
console.log(letters);            // ["a", "b", "c"]

letters.reverse();
console.log(letters);            // ["c", "b", "a"]

// ⚠ Numbers sort as TEXT by default — always give a comparator
let nums = [10, 5, 40, 25];
nums.sort();                     // [10, 25, 40, 5]  ← wrong!
console.log(nums);

nums.sort((a, b) => a - b);      // ascending
console.log(nums);               // [5, 10, 25, 40]

nums.sort((a, b) => b - a);      // descending
console.log(nums);               // [40, 25, 10, 5]

// join turns the array into a string
console.log(["a", "b", "c"].join("-"));   // a-b-c

// concat merges arrays
console.log([1, 2].concat([3, 4]));       // [1, 2, 3, 4]
```

Classic bug

Without a comparator, `sort()` compares elements as strings, so `10` comes before `5`. Always pass `(a, b) => a - b` when sorting numbers.

OTHER USEFUL ARRAY TOOLS

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method / Property	What it does
`Array.isArray(x)`	Checks if `x` is really an array
`toString()`	Converts array to comma-separated string
`fill(value)`	Replaces every element with the same value
`flat()`	Flattens nested arrays into one
`Array.from("abc")`	Builds an array from any iterable
`Array.of(1, 2, 3)`	Builds an array from arguments

JAVASCRIPT — UTILITY METHODS

```
console.log(Array.isArray([1, 2]));     // true
console.log(Array.isArray("1,2"));      // false

console.log([1, 2, 3].toString());      // "1,2,3"

let zeros = new Array(4).fill(0);
console.log(zeros);                     // [0, 0, 0, 0]

let nested = [1, [2, [3, [4]]]];
console.log(nested.flat(Infinity));     // [1, 2, 3, 4]

console.log(Array.from("hello"));       // ["h", "e", "l", "l", "o"]
```

2.7 Two-dimensional arrays

JAVASCRIPT — 2D ARRAYS

```
let grid = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(grid[1][2]);        // 6  (row 1, column 2)

// Loop through every cell
for (let row = 0; row < grid.length; row++) {
  for (let col = 0; col < grid[row].length; col++) {
    console.log(grid[row][col]);
  }
}
```

2.8 Arrays are reference types

JAVASCRIPT — COPYING AN ARRAY

```
let a = [1, 2, 3];
let b = a;              // b points to the SAME array

b.push(4);
console.log(a);         // [1, 2, 3, 4]  ← a also changed!

// ✅ Real copies
let c = [...a];         // spread operator
let d = a.slice();      // slice with no arguments
let e = Array.from(a);

c.push(99);
console.log(a);         // [1, 2, 3, 4]  (unchanged)
```

Remember

Arrays and objects are **reference types**. Assigning one variable to another does not copy the data — both names point to the same memory location.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3 User Defined Functions

3.1 What is a function?

A **function** is a reusable block of code that performs a specific task. You write it once and call it as many times as you like. Functions make code shorter, cleaner and easier to fix.

> **Analogy**
>
> A function is like a recipe. You write the recipe once, and every time you want the dish you just follow it — you don't rewrite it from scratch.

Why use functions?

- **Reusability** — write once, call many times.
- **Readability** — meaningful names explain what the code does.
- **Maintainability** — fix a bug in one place, not ten.
- **Testing** — small functions are easy to test separately.

3.2 Function declaration (the standard way)

JAVASCRIPT — SYNTAX

```
function functionName(parameter1, parameter2) {
  // code to run
  return result;          // optional
}

functionName(argument1, argument2);   // calling the function
```

JAVASCRIPT — SIMPLE FUNCTION

```
function greet() {
  console.log("Hello, welcome!");
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
}

greet();      // Hello, welcome!
greet();      // Hello, welcome!
```

3.3 Parameters and arguments

Parameters are the names written in the function definition. **Arguments** are the actual values you pass when calling the function.

JAVASCRIPT — PARAMETERS

```
// "name" and "age" are parameters
function introduce(name, age) {
  console.log(`My name is ${name} and I am ${age} years old.`);
}

// "Riya" and 20 are arguments
introduce("Riya", 20);
introduce("Amit", 25);
```

▸ OUTPUT

```
My name is Riya and I am 20 years old.
My name is Amit and I am 25 years old.
```

3.4 The return statement

`return` sends a value back to wherever the function was called, and it immediately ends the function.

JAVASCRIPT — RETURN

```
function add(a, b) {
  return a + b;
}

let result = add(5, 3);
console.log(result);        // 8
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// You can use the return value directly
console.log(add(10, 20));   // 30
```

return vs console.log

`console.log()` only *prints* a value. `return` actually *gives* the value back so you can store it or use it in further calculations.

JAVASCRIPT — A FUNCTION STOPS AT RETURN

```
function checkAge(age) {
  if (age < 18) {
    return "Minor";       // function ends here
  }
  return "Adult";         // only reached if age >= 18
}

console.log(checkAge(15));   // Minor
console.log(checkAge(22));   // Adult
```

If there is no return…

A function without a `return` statement automatically returns `undefined`.

3.5 Default parameters

JAVASCRIPT — DEFAULT VALUES

```
function greet(name = "Guest") {
  console.log("Hello, " + name);
}

greet("Riya");   // Hello, Riya
greet();         // Hello, Guest   (default used)
```

3.6 Rest parameters (...)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Collects any number of arguments into a real array.

JAVASCRIPT — REST PARAMETERS

```
function sumAll(...numbers) {
  let total = 0;
  for (let n of numbers) {
    total += n;
  }
  return total;
}

console.log(sumAll(1, 2, 3));          // 6
console.log(sumAll(10, 20, 30, 40));   // 100
```

3.7 Function expressions

A function can be stored in a variable. This is called a **function expression**.

JAVASCRIPT — FUNCTION EXPRESSION

```
const multiply = function (a, b) {
  return a * b;
};

console.log(multiply(4, 5));   // 20
```

3.8 Arrow functions (modern shorthand)

JAVASCRIPT — ARROW FUNCTIONS

```
// Full syntax
const add = (a, b) => {
  return a + b;
};

// One-line version — "return" is automatic
const add2 = (a, b) => a + b;
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// Single parameter — brackets optional
const square = x => x * x;

// No parameters
const sayHi = () => console.log("Hi!");

console.log(add(2, 3));      // 5
console.log(square(6));      // 36
```

Aspect	Function declaration	Arrow function
Hoisted (usable before definition)	Yes	No
Own `this`	Yes	No (inherits from outer scope)
Syntax length	Longer	Shorter
Best for	General purpose, main logic	Short callbacks, array methods

3.9 Anonymous and IIFE functions

JAVASCRIPT — ANONYMOUS & IIFE

```
// Anonymous function (no name) — often used as a callback
setTimeout(function () {
  console.log("Runs after 1 second");
}, 1000);

// IIFE — Immediately Invoked Function Expression
(function () {
  console.log("I run immediately!");
})();

// Arrow IIFE
(() => {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  console.log("Me too!");
})();
```

3.10 Functions calling other functions

JAVASCRIPT — COMPOSITION

```
function areaOfCircle(r) {
  return 3.14159 * r * r;
}

function costOfCircle(r, pricePerUnit) {
  return areaOfCircle(r) * pricePerUnit;   // calling another functi
}

console.log(costOfCircle(5, 10));   // 785.3975
```

3.11 Callback functions (brief introduction)

A **callback** is a function passed as an argument to another function, to be called later.

JAVASCRIPT — CALLBACKS

```
function processUser(name, callback) {
  console.log("Processing " + name);
  callback(name);
}

function done(name) {
  console.log("Finished with " + name);
}

processUser("Riya", done);

// Often written inline with an arrow function
processUser("Amit", (n) => console.log("Done: " + n));
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3.12 Scope inside functions

Variables declared inside a function with `let` or `const` are **local** — they cannot be used outside that function.

JAVASCRIPT — LOCAL VS GLOBAL

```
let globalVar = "I am global";

function demo() {
  let localVar = "I am local";
  console.log(globalVar);   // ✅ accessible
  console.log(localVar);    // ✅ accessible
}

demo();
// console.log(localVar);   ❌ ReferenceError
```

> **Best practice**
>
> Keep variables as local as possible. Global variables can be changed from anywhere, which makes bugs very hard to find.

3.13 Complete example — a mini calculator

JAVASCRIPT

```
function add(a, b)      { return a + b; }
function subtract(a, b) { return a - b; }
function multiply(a, b) { return a * b; }
function divide(a, b) {
  if (b === 0) {
    return "Cannot divide by zero";
  }
  return a / b;
}

function calculate(num1, num2, operation) {
  switch (operation) {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    case "+": return add(num1, num2);
    case "-": return subtract(num1, num2);
    case "*": return multiply(num1, num2);
    case "/": return divide(num1, num2);
    default:  return "Invalid operation";
  }
}

console.log(calculate(10, 5, "+"));   // 15
console.log(calculate(10, 5, "/"));   // 2
console.log(calculate(10, 0, "/"));   // Cannot divide by zero
```

4 Built-in Functions: String, Math, Array, Date

JavaScript comes with many ready-made objects and functions. You do not need to write them yourself — you just call them. This chapter covers the four most commonly used ones.

4.1 Global functions (bonus)

Function	Purpose	Example → Result
`Number(x)`	Convert to number	`Number("42")` → `42`
`parseInt(x)`	Convert to whole number	`parseInt("42px")` → `42`
`parseFloat(x)`	Convert to decimal number	`parseFloat("3.14abc")` → `3.14`
`String(x)`	Convert to string	`String(25)` → `"25"`
`isNaN(x)`	Is it "Not a Number"?	`isNaN("abc")` → `true`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Function	Purpose	Example → Result
`Boolean(x)`	Convert to true / false	`Boolean(0)` → `false`

4.2 String functions and properties

Strings are immutable — string methods always return a *new* string; they never change the original one.

Method / Property	Description	Example → Result
`length`	Number of characters (property, not a method)	`"Hello".length` → `5`
`toUpperCase()`	Convert to capital letters	`"hi".toUpperCase()` → `"HI"`
`toLowerCase()`	Convert to small letters	`"HI".toLowerCase()` → `"hi"`
`trim()`	Remove spaces from both ends	`" hi ".trim()` → `"hi"`
`charAt(i)`	Character at index i	`"Hello".charAt(1)` → `"e"`
`charCodeAt(i)`	Unicode number of character	`"A".charCodeAt(0)` → `65`
`indexOf(s)`	First position of substring, or -1	`"Hello".indexOf("l")` → `2`
`lastIndexOf(s)`	Last position	`"Hello".lastIndexOf("l")`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method / Property	Description	Example → Result
	of substring	→ `3`
`includes(s)`	Does it contain the substring?	`"Hello".includes("ell")` → `true`
`startsWith(s)`	Does it begin with s?	`"Hello".startsWith("He")` → `true`
`endsWith(s)`	Does it end with s?	`"Hello".endsWith("lo")` → `true`
`slice(a, b)`	Extract from a up to (not including) b	`"Hello".slice(1, 4)` → `"ell"`
`substring(a, b)`	Similar to slice (no negatives)	`"Hello".substring(1, 4)` → `"ell"`
`substr(a, len)`	Start at a, take len characters	`"Hello".substr(1, 3)` → `"ell"`
`replace(a, b)`	Replace first occurrence	`"a-b-c".replace("-", "+")` → `"a+b-c"`
`replaceAll(a, b)`	Replace every occurrence	`"a-b-c".replaceAll("-", "+")` → `"a+b+c"`
`split(sep)`	Turn string into array	`"a,b,c".split(",")` → `["a","b","c"]`
`concat(s)`	Join strings	`"Hi".concat(" there")` → `"Hi there"`
`repeat(n)`	Repeat the string n times	`"ab".repeat(3)` → `"ababab"`
`padStart(n, c)`	Pad at the beginning	`"5".padStart(3, "0")` → `"005"`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method / Property	Description	Example → Result
`padEnd(n, c)`	Pad at the end	`"5".padEnd(3, "x")` → `"5xx"`
`toString()`	Convert to string	`(255).toString()` → `"255"`
`String.fromCharCode(n)`	Build a character from a code	`String.fromCharCode(65)` → `"A"`

JAVASCRIPT — STRING EXAMPLES

```
let text = "  JavaScript is Fun  ";

console.log(text.length);                // 21 (spaces counted)
console.log(text.trim());                // "JavaScript is Fun"
console.log(text.trim().toUpperCase()); // "JAVASCRIPT IS FUN"

let email = "User@Example.COM";
console.log(email.toLowerCase());       // "user@example.com"

let sentence = "I love JavaScript";
console.log(sentence.includes("love"));     // true
console.log(sentence.indexOf("JavaScript")); // 7
console.log(sentence.slice(7));             // "JavaScript"
console.log(sentence.replace("love", "like")); // "I like JavaScript

let csv = "apple,banana,mango";
let arr = csv.split(",");
console.log(arr);                        // ["apple", "banana", "mango

console.log("5".padStart(4, "0"));      // "0005"
console.log("*".repeat(10));            // "**********"
```

Practical tip

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

> To check a string regardless of capitalisation, always convert it to lowercase first: `str.toLowerCase().includes("admin")`.

4.3 Math object

`Math` is a built-in object with properties and methods for mathematical operations. You never create it — you use it directly.

CONSTANTS

Property	Value	Meaning
`Math.PI`	3.141592653589793	Ratio of circle circumference to diameter
`Math.E`	2.718281828459045	Euler's number
`Math.SQRT2`	1.4142135623730951	Square root of 2
`Math.LN2`	0.6931471805599453	Natural log of 2

METHODS

Method	Description	Example → Result
`Math.abs(x)`	Absolute (positive) value	`Math.abs(-7)` → `7`
`Math.ceil(x)`	Round **up**	`Math.ceil(4.1)` → `5`
`Math.floor(x)`	Round **down**	`Math.floor(4.9)` → `4`
`Math.round(x)`	Round to nearest integer	`Math.round(4.5)` → `5`
`Math.trunc(x)`	Remove the decimal part	`Math.trunc(4.9)` → `4`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method	Description	Example → Result
`Math.max(a, b, …)`	Largest value	`Math.max(3, 9, 2)` → `9`
`Math.min(a, b, …)`	Smallest value	`Math.min(3, 9, 2)` → `2`
`Math.pow(a, b)`	a to the power b	`Math.pow(2, 5)` → `32`
`Math.sqrt(x)`	Square root	`Math.sqrt(81)` → `9`
`Math.cbrt(x)`	Cube root	`Math.cbrt(27)` → `3`
`Math.random()`	Random number from 0 (inclusive) to 1 (exclusive)	`Math.random()` → `0.7345…`
`Math.sign(x)`	-1, 0 or 1 depending on sign	`Math.sign(-8)` → `-1`
`Math.hypot(a, b)`	Square root of sum of squares	`Math.hypot(3, 4)` → `5`
`Math.log(x)`	Natural logarithm	`Math.log(Math.E)` → `1`

JAVASCRIPT — MATH EXAMPLES

```
console.log(Math.PI);                 // 3.141592653589793

console.log(Math.abs(-15));           // 15
console.log(Math.ceil(4.01));         // 5
console.log(Math.floor(4.99));        // 4
console.log(Math.round(4.5));         // 5
console.log(Math.round(4.4));         // 4

console.log(Math.max(10, 45, 22));    // 45
console.log(Math.min(10, 45, 22));    // 10
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(Math.pow(3, 4));          // 81
console.log(Math.sqrt(144));          // 12
```

GENERATING RANDOM NUMBERS (VERY USEFUL)

JAVASCRIPT — RANDOM HELPERS

```
// Random decimal between 0 and 1
console.log(Math.random());

// Random integer from 0 to 9
let a = Math.floor(Math.random() * 10);
console.log(a);

// Random integer from 1 to 6 (like a dice)
let dice = Math.floor(Math.random() * 6) + 1;
console.log(dice);

// Random integer between min and max (inclusive)
function randomBetween(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(randomBetween(1, 100));   // e.g. 57
```

Formula to memorise

Random integer between *min* and *max* (both included):

`Math.floor(Math.random() * (max - min + 1)) + min`

ROUNDING MONEY TO 2 DECIMALS

JAVASCRIPT

```
let price = 1234.5678;

// Method 1
let rounded = Math.round(price * 100) / 100;
console.log(rounded);                    // 1234.57
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// Method 2 — easier for display
console.log(price.toFixed(2));          // "1234.57"   (a string!)
```

4.4 Array built-in functions (recap)

These were covered in Chapter 2. Here is a compact revision table:

Category	Methods
Add / remove	`push` , `pop` , `shift` , `unshift` , `splice` , `slice`
Search	`indexOf` , `lastIndexOf` , `includes` , `find` , `findIndex`
Transform	`map` , `filter` , `reduce` , `flat` , `fill`
Test	`some` , `every` , `Array.isArray`
Order	`sort` , `reverse`
Convert	`join` , `toString` , `concat` , `Array.from`
Iterate	`forEach` , `for...of`

4.5 Date object

The `Date` object works with dates and times. Unlike `Math` , you must create an instance with the `new` keyword.

CREATING DATES

JAVASCRIPT — CREATING DATE OBJECTS

```
// 1. Current date and time
let now = new Date();
console.log(now);

// 2. From a date string
let d1 = new Date("2024-12-25");
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(d1);

// 3. From year, month, day, hour, minute, second
//    ⚠ Month is ZERO-based: 0 = January, 11 = December
let d2 = new Date(2024, 11, 25, 10, 30, 0);
console.log(d2);          // 25 December 2024, 10:30:00

// 4. From milliseconds since 1 Jan 1970
let d3 = new Date(0);
console.log(d3);          // 01 January 1970, 00:00:00 UTC
```

Very common bug

Months in JavaScript start at **0**. So `new Date(2024, 0, 1)` is 1 January 2024, and `new Date(2024, 11, 25)` is 25 December 2024.

GETTING PARTS OF A DATE

Method	Returns	Range
`getFullYear()`	Year	e.g. 2024
`getMonth()`	Month	0–11
`getDate()`	Day of the month	1–31
`getDay()`	Day of the week	0 (Sunday) – 6 (Saturday)
`getHours()`	Hours	0–23
`getMinutes()`	Minutes	0–59
`getSeconds()`	Seconds	0–59
`getMilliseconds()`	Milliseconds	0–999
`getTime()`	Milliseconds since 1 Jan 1970	large number

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — READING DATE PARTS

```
let now = new Date("2024-12-25T10:30:45");

console.log(now.getFullYear());    // 2024
console.log(now.getMonth());       // 11   (December — zero based)
console.log(now.getDate());        // 25
console.log(now.getDay());         // 3    (Wednesday)
console.log(now.getHours());       // 10
console.log(now.getMinutes());     // 30
console.log(now.getSeconds());     // 45

// Human-friendly month name
const months = ["Jan","Feb","Mar","Apr","May","Jun",
                "Jul","Aug","Sep","Oct","Nov","Dec"];
console.log(months[now.getMonth()]);   // "Dec"
```

SETTING PARTS OF A DATE

JAVASCRIPT — SET METHODS

```
let d = new Date();

d.setFullYear(2030);
d.setMonth(5);          // June
d.setDate(15);
d.setHours(9, 0, 0);    // 9:00:00

console.log(d);
```

CONVERTING A DATE TO A STRING

Method	Output style
`toString()`	Full readable date and time
`toDateString()`	Date part only
`toTimeString()`	Time part only

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method	Output style
`toLocaleDateString()`	Date in local format (e.g. 25/12/2024)
`toLocaleTimeString()`	Time in local format
`toISOString()`	ISO 8601 format, e.g. 2024-12-25T10:30:45.000Z

JAVASCRIPT — DATE FORMATTING

```
let d = new Date("2024-12-25T10:30:45");

console.log(d.toString());
console.log(d.toDateString());         // Wed Dec 25 2024
console.log(d.toLocaleDateString());   // 25/12/2024 (depends on loca
console.log(d.toLocaleTimeString());   // 10:30:45
console.log(d.toISOString());          // 2024-12-25T10:30:45.000Z
```

STATIC DATE METHODS AND DATE ARITHMETIC

JAVASCRIPT — DATE.NOW() AND DIFFERENCES

```
// Current time in milliseconds (no object needed)
console.log(Date.now());

// Difference between two dates in days
let start = new Date("2024-01-01");
let end   = new Date("2024-12-31");

let diffMs   = end - start;                        // milliseconds
let diffDays = diffMs / (1000 * 60 * 60 * 24);     // days

console.log(Math.round(diffDays));   // 365

// Age calculator
function getAge(birthDate) {
  let today = new Date();
  let birth = new Date(birthDate);
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  let age = today.getFullYear() - birth.getFullYear();
  let m = today.getMonth() - birth.getMonth();

  if (m < 0 || (m === 0 && today.getDate() < birth.getDate())) {
    age--;
  }
  return age;
}

console.log(getAge("2004-05-15"));   // e.g. 20
```

Useful

Subtracting two dates gives the difference in **milliseconds**. Divide by `1000` for seconds, `60000` for minutes, `3600000` for hours, and `86400000` for days.

5 Events

5.1 What is an event?

An **event** is an action or occurrence that happens in the browser — the user clicks a button, types a key, moves the mouse, submits a form, or the page finishes loading. JavaScript can "listen" for these events and run code in response.

Analogy

An event is like a doorbell. When someone presses it (the event fires), you react by opening the door (your event handler runs).

5.2 Three ways to attach an event handler

METHOD 1 — HTML ATTRIBUTE (INLINE)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

HTML

```
<button onclick="showMessage()">Click Me</button>

<script>
  function showMessage() {
    alert("Button clicked!");
  }
</script>
```

Simple, but mixes HTML with JavaScript. Not recommended for larger projects.

METHOD 2 — DOM PROPERTY

HTML + JAVASCRIPT

```
<button id="myBtn">Click Me</button>

<script>
  const btn = document.getElementById("myBtn");

  btn.onclick = function () {
    alert("Button clicked!");
  };
</script>
```

Better separation, but you can attach only *one* handler per event this way.

METHOD 3 — ADDEVENTLISTENER() ✅ RECOMMENDED

HTML + JAVASCRIPT

```
<button id="myBtn">Click Me</button>

<script>
  const btn = document.getElementById("myBtn");

  btn.addEventListener("click", function () {
    alert("Button clicked!");
  });
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  // You can attach MULTIPLE handlers
  btn.addEventListener("click", () => console.log("Second handler"))
</script>
```

Approach	Multiple handlers?	Recommended
Inline `onclick=""`	No	❌ Only for quick tests
`element.onclick = fn`	No (overwrites)	⚠️ Sometimes
`addEventListener()`	Yes	✅ Best practice

5.3 The event object

When an event fires, the browser passes an **event object** to your handler. It contains useful information about what happened.

JAVASCRIPT — THE EVENT OBJECT

```
document.getElementById("myBtn").addEventListener("click", function
  console.log(e.type);          // "click"
  console.log(e.target);        // the element that was clicked
  console.log(e.clientX, e.clientY);   // mouse coordinates
});
```

5.4 The complete list of required events

Event	Fires when…	Typical use
`onclick`	The element is clicked once	Buttons, links, toggles
`ondblclick`	The element is double-clicked	Rename, open details

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Event	Fires when…	Typical use
`onmouseover`	The mouse enters the element	Highlight, show tooltip
`onmouseout`	The mouse leaves the element	Remove highlight
`onkeypress`	A key is pressed and released (deprecated but still asked in exams)	Detect typed character
`onkeyup`	A key is released	Live search, character count
`onfocus`	An input gains focus	Highlight field, show hint
`onblur`	An input loses focus	Validate field after typing
`onload`	The page (or image) finishes loading	Start-up code, preloaders
`onchange`	An input's value changed and focus left it	Dropdowns, checkbox logic
`onsubmit`	A form is submitted	Form validation
`onreset`	A form's reset button is clicked	Confirm clearing the form

5.5 Each event explained with an example

ONCLICK

```HTML
<button onclick="sayHi()">Say Hi</button>

<script>
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  function sayHi() {
    alert("Hi there!");
  }
</script>
```

ONDBLCLICK

HTML

```
<p ondblclick="this.style.color='red'">
  Double-click me to turn red!
</p>
```

ONMOUSEOVER AND ONMOUSEOUT

HTML

```
<div id="box"
     onmouseover="this.style.background='#4f46e5'; this.style.color=
     onmouseout="this.style.background=''; this.style.color='';">
  Hover over me!
</div>
```

ONKEYPRESS AND ONKEYUP

HTML + JAVASCRIPT

```
<input type="text" id="nameInput" placeholder="Type your name" />
<p id="preview"></p>

<script>
  const input = document.getElementById("nameInput");
  const preview = document.getElementById("preview");

  // onkeyup — fires every time a key is released
  input.addEventListener("keyup", function (e) {
    preview.textContent = "You typed: " + this.value;
    console.log("Key code: " + e.keyCode);
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  });

  // onkeypress — fires when a character key is pressed
  input.addEventListener("keypress", function (e) {
    console.log("Character: " + String.fromCharCode(e.charCode));
  });
</script>
```

ONFOCUS AND ONBLUR

HTML

```
<input type="text"
       placeholder="Click inside me"
       onfocus="this.style.background='#eef2ff'; this.style.border='
       onblur="this.style.background=''; this.style.border='';" />
```

JAVASCRIPT — VALIDATING ON BLUR

```
const email = document.getElementById("email");

email.addEventListener("blur", function () {
  if (!this.value.includes("@")) {
    alert("Please enter a valid email address.");
  }
});
```

ONLOAD

HTML

```
<body onload="init()">

<script>
  function init() {
    console.log("Page fully loaded — safe to access elements.");
    alert("Welcome!");
  }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
</script>

</body>
```

Alternative

Instead of `onload` on the body, you can put your `<script>` tag at the end of the body, or use `document.addEventListener("DOMContentLoaded", …)`.

ONCHANGE

HTML

```
<select id="course" onchange="showCourse()">
  <option value="">-- Select --</option>
  <option value="BCA">BCA</option>
  <option value="MCA">MCA</option>
  <option value="BBA">BBA</option>
</select>

<p id="result"></p>

<script>
  function showCourse() {
    const select = document.getElementById("course");
    document.getElementById("result").textContent =
      "You selected: " + select.value;
  }
</script>
```

ONSUBMIT AND ONRESET

HTML

```
<form onsubmit="return validateForm(event)" onreset="return confirmR
  <input type="text" id="username" placeholder="Username" />
  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
<script>
  function validateForm(e) {
    const username = document.getElementById("username").value;

    if (username.trim() === "") {
      alert("Username cannot be empty!");
      e.preventDefault();       // stop the form from submitting
      return false;             // also stops submission (older styl
    }

    alert("Form submitted successfully!");
    return true;
  }

  function confirmReset() {
    return confirm("Clear all fields?");
  }
</script>
```

Two ways to stop submission

`e.preventDefault()` is the modern way. `return false` from an inline handler also works. Use one of them, not both in confusion.

5.6 Event bubbling in one minute

When an event happens on a child element, it "bubbles up" and also triggers handlers on its parent elements. `e.stopPropagation()` stops this.

JAVASCRIPT — BUBBLING

```
document.getElementById("child").addEventListener("click", function
  console.log("Child clicked");
  // e.stopPropagation();   // uncomment to stop bubbling
});

document.getElementById("parent").addEventListener("click", function
  console.log("Parent clicked");
});
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// Clicking the child prints BOTH lines (child, then parent)
```

5.7 Removing an event listener

JAVASCRIPT

```
function handleClick() {
  console.log("Clicked once!");
}

const btn = document.getElementById("btn");

btn.addEventListener("click", handleClick);

// Later, remove it (the function reference must be the same)
btn.removeEventListener("click", handleClick);
```

6 DOM & History Object

6.1 What is the DOM?

The **DOM (Document Object Model)** is a tree-like representation of your HTML page that JavaScript can read and change. Every HTML tag becomes a *node* (an object) in this tree.

> **Key idea**
>
> JavaScript does not change your HTML file. It changes the **DOM** — the live model the browser is displaying. That is why the page updates instantly without a reload.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
HTML → DOM TREE

<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>World</p>
  </body>
</html>

Document
 └── html
      ├── head
      │    └── title → "My Page"
      └── body
           ├── h1 → "Hello"
           └── p  → "World"
```

The entry point is always the global `document` object.

6.2 Selecting elements

Method	Returns	Example
`getElementById(id)`	One element (or `null`)	`document.getElementById`
`getElementsByClassName(cls)`	Live HTMLCollection	`document.getElementsBy`
`getElementsByTagName(tag)`	Live HTMLCollection	`document.getElementsBy`
`querySelector(css)`	First matching element	`document.querySelector(`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method	Returns	Example
`querySelectorAll(css)`	Static NodeList (all matches)	`document.querySelectorA`

HTML

```
<p id="para">Original text</p>
<ul>
  <li class="item">One</li>
  <li class="item">Two</li>
  <li class="item">Three</li>
</ul>
```

JAVASCRIPT — SELECTING

```
// Single element
const para = document.getElementById("para");
console.log(para.textContent);     // "Original text"

// All items by class
const items = document.getElementsByClassName("item");
console.log(items.length);         // 3
console.log(items[0].textContent); // "One"

// CSS selector — first match
const first = document.querySelector(".item");
console.log(first.textContent);    // "One"

// CSS selector — all matches
const all = document.querySelectorAll(".item");
all.forEach(li => console.log(li.textContent));
// One
// Two
// Three
```

Recommendation

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

`querySelector()` and `querySelectorAll()` accept any CSS selector, which makes them the most flexible choice.

6.3 Reading and changing content

Property	What it does
`innerHTML`	Gets/sets HTML content (tags are interpreted)
`innerText`	Gets/sets visible text only
`textContent`	Gets/sets all text (including hidden), faster and safer
`value`	Gets/sets the value of form fields

JAVASCRIPT — CHANGING CONTENT

```
const box = document.getElementById("box");

box.innerHTML = "<strong>Bold text</strong>";   // HTML is parsed
box.textContent = "<strong>Plain text</strong>"; // shown literally

// Form field
const input = document.getElementById("name");
console.log(input.value);        // read
input.value = "New value";       // write
```

Security warning

Never put raw user input into `innerHTML` — it can execute malicious scripts (XSS). Use `textContent` when displaying user data.

6.4 Changing styles and classes

JAVASCRIPT — STYLE AND CLASSLIST

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
const box = document.getElementById("box");

// Inline styles (camelCase property names)
box.style.color = "white";
box.style.backgroundColor = "#4f46e5";
box.style.padding = "20px";
box.style.fontSize = "18px";

// classList — the clean way
box.classList.add("highlight");
box.classList.remove("highlight");
box.classList.toggle("active");          // add if absent, remove if
console.log(box.classList.contains("active"));   // true / false
```

6.5 Attributes

JAVASCRIPT — ATTRIBUTES

```
const link = document.getElementById("myLink");

console.log(link.getAttribute("href"));
link.setAttribute("href", "https://example.com");
link.removeAttribute("target");

// Common shortcuts
const img = document.querySelector("img");
img.src = "new.jpg";
img.alt = "New image";

const btn = document.querySelector("button");
btn.disabled = true;
```

6.6 Creating and removing elements

JAVASCRIPT — CREATEELEMENT / APPENDCHILD / REMOVE

```
// 1. Create the element
const newLi = document.createElement("li");
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// 2. Give it content and attributes
newLi.textContent = "Four";
newLi.className = "item";
newLi.id = "item-four";

// 3. Find the parent and attach it
const list = document.querySelector("ul");
list.appendChild(newLi);              // adds at the end

// 4. Remove it later
newLi.remove();                       // modern way
// list.removeChild(newLi);           // older way

// 5. Other insertion methods
const firstLi = list.firstElementChild;
list.insertBefore(newLi, firstLi);    // insert before the first ite

// 6. Replace an element
const replacement = document.createElement("li");
replacement.textContent = "Replaced";
list.replaceChild(replacement, firstLi);
```

6.7 Navigating the DOM tree

Property	Meaning
`parentElement`	The parent element
`children`	All child elements
`firstElementChild`	First child element
`lastElementChild`	Last child element
`nextElementSibling`	Next element at the same level
`previousElementSibling`	Previous element at the same level

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

JAVASCRIPT — TRAVERSING

```
const second = document.querySelectorAll(".item")[1];

console.log(second.parentElement.tagName);      // UL
console.log(second.nextElementSibling.textContent);  // "Three"
console.log(second.previousElementSibling.textContent); // "One"
```

6.8 The History object

`window.history` lets JavaScript interact with the browser's session history — the list of pages the user has visited in this tab.

Member	Description
`history.length`	Number of entries in the history list
`history.back()`	Go to the previous page (same as the Back button)
`history.forward()`	Go to the next page (same as Forward)
`history.go(n)`	Go n steps: `-1` back, `1` forward, `0` reload
`history.pushState(state, title, url)`	Add a new entry without reloading
`history.replaceState(state, title, url)`	Replace the current entry
`history.state`	The state object stored with the current entry

JAVASCRIPT — HISTORY BASICS

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// Go back one page
function goBack() {
  history.back();
}

// Go forward one page
function goForward() {
  history.forward();
}

// Go back 2 pages
function goBackTwo() {
  history.go(-2);
}

// Reload the current page
function refresh() {
  history.go(0);
}

// How many pages in this tab's history
console.log(history.length);
```

HTML — HISTORY BUTTONS

```
<button onclick="history.back()">← Back</button>
<button onclick="history.forward()">Forward →</button>
<button onclick="history.go(-1)">Previous</button>
```

PUSHSTATE AND REPLACESTATE (SINGLE-PAGE APPS)

JAVASCRIPT — PUSHSTATE

```
// Add a history entry WITHOUT reloading the page
history.pushState({ page: "about" }, "", "/about");

// Replace the current entry instead of adding one
history.replaceState({ page: "home" }, "", "/home");
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
// Read the stored state
console.log(history.state);   // { page: "about" }

// React when the user presses Back / Forward
window.addEventListener("popstate", function (e) {
  console.log("Navigation changed. State:", e.state);
});
```

> **Important limitation**
>
> The History API works with **real web addresses**. If you open an HTML file directly from your computer (`file:///…`), `pushState` will throw a security error. Use a local server or test `back()` / `forward()` on real pages.

6.9 History vs Location

Object	Purpose	Example
`history`	Navigate between visited pages	`history.back()`
`location`	Go to a specific new URL / reload	`location.href = "page.html"`

JAVASCRIPT — LOCATION OBJECT

```
console.log(location.href);       // full URL
console.log(location.hostname);   // domain
console.log(location.pathname);   // path
console.log(location.search);     // query string
console.log(location.hash);       // #fragment

// Navigate
location.href = "https://example.com";
location.assign("page2.html");
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
location.replace("page3.html");   // does NOT add a history entry
location.reload();                // refresh
```

7 Form Validation & E-mail Validation

7.1 What is form validation?

Form validation is the process of checking that the data entered by the user is complete, correctly formatted and sensible *before* it is sent to the server.

Why it matters:

- Prevents empty or incomplete submissions.
- Stops wrong data types (letters in a phone number).
- Gives the user instant feedback instead of a slow server response.
- Reduces load on the server and the database.
- Improves security by blocking obvious bad input.

> **Remember**
>
> Client-side validation is for *user convenience*. It can be bypassed, so the server must always validate again.

7.2 Two levels of validation

Type	Where it runs	Example
HTML5 validation	Built into the browser	`required`, `type="email"`, `minlength`, `pattern`
JavaScript	Your own code	Custom rules, regex, error messages

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Type	Where it runs	Example
validation		

7.3 HTML5 validation attributes

Attribute	Purpose	Example
`required`	Field must not be empty	`<input required>`
`type="email"`	Basic email format check	`<input type="email">`
`type="number"`	Only numbers allowed	`<input type="number">`
`minlength`	Minimum characters	`minlength="6"`
`maxlength`	Maximum characters	`maxlength="30"`
`min` / `max`	Range for numbers	`min="1" max="100"`
`pattern`	Must match a regular expression	`pattern="[0-9]{10}"`
`placeholder`	Hint text inside the field	`placeholder="Enter name"`

HTML — HTML5 VALIDATION

```
<form>
  <input type="text" name="username" required minlength="3" maxlengt
         placeholder="Username" />

  <input type="email" name="email" required placeholder="you@example

  <input type="tel" name="phone" pattern="[0-9]{10}" required
         placeholder="10-digit phone" />

  <input type="number" name="age" min="1" max="120" required />
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  <button type="submit">Submit</button>
</form>
```

If any rule fails, the browser shows its own message and the form is not submitted.

7.4 JavaScript form validation — the onsubmit event

Attach a function to the form's `onsubmit` event. If it returns `true`, the form submits; if it returns `false`, submission is cancelled.

HTML — A SIMPLE REGISTRATION FORM

```
<form id="regForm" onsubmit="return validateForm(event)">

  <label>Full Name:</label>
  <input type="text" id="fullname" />
  <span id="nameError" class="error"></span>

  <label>Email:</label>
  <input type="text" id="email" />
  <span id="emailError" class="error"></span>

  <label>Password:</label>
  <input type="password" id="password" />
  <span id="passError" class="error"></span>

  <label>Age:</label>
  <input type="text" id="age" />
  <span id="ageError" class="error"></span>

  <button type="submit">Register</button>
  <button type="reset">Reset</button>

</form>
```

JAVASCRIPT — THE VALIDATION FUNCTION

```
function validateForm(e) {
  // Clear previous error messages
  document.getElementById("nameError").textContent  = "";
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    document.getElementById("emailError").textContent = "";
    document.getElementById("passError").textContent  = "";
    document.getElementById("ageError").textContent   = "";

    let isValid = true;

    // ---- 1. Name ----
    const name = document.getElementById("fullname").value.trim();

    if (name === "") {
      document.getElementById("nameError").textContent = "Name is requ
      isValid = false;
    } else if (name.length < 3) {
      document.getElementById("nameError").textContent =
        "Name must be at least 3 characters.";
      isValid = false;
    }

    // ---- 2. Email ----
    const email = document.getElementById("email").value.trim();

    if (email === "") {
      document.getElementById("emailError").textContent = "Email is re
      isValid = false;
    } else if (!isValidEmail(email)) {
      document.getElementById("emailError").textContent =
        "Please enter a valid email address.";
      isValid = false;
    }

    // ---- 3. Password ----
    const password = document.getElementById("password").value;

    if (password.length < 6) {
      document.getElementById("passError").textContent =
        "Password must be at least 6 characters.";
      isValid = false;
    }

    // ---- 4. Age ----
    const age = document.getElementById("age").value.trim();
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
  if (age === "") {
    document.getElementById("ageError").textContent = "Age is requir
    isValid = false;
  } else if (isNaN(age)) {
    document.getElementById("ageError").textContent = "Age must be a
    isValid = false;
  } else if (Number(age) < 18 || Number(age) > 120) {
    document.getElementById("ageError").textContent =
      "Age must be between 18 and 120.";
    isValid = false;
  }

  // ---- Final decision ----
  if (!isValid) {
    e.preventDefault();     // stop submission
    return false;
  }

  alert("Form submitted successfully!");
  return true;
}
```

CSS — STYLING THE ERROR MESSAGES

```
.error {
  color: #ef4444;
  font-size: 0.85rem;
  display: block;
  margin-top: 4px;
}
```

7.5 Common validation checks

Check	How to test it
Field is not empty	`value.trim() === ""`
Minimum length	`value.length >= 6`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Check	How to test it
Maximum length	`value.length <= 30`
Only letters	`/^[A-Za-z\s]+$/.test(value)`
Only digits	`/^\d+$/.test(value)`
Is a valid number	`!isNaN(value)`
Within a range	`n >= 1 && n <= 100`
Valid email	Regex test (see below)
Password strength	Length + uppercase + number + symbol
Passwords match	`pass1 === pass2`

7.6 E-mail validation

There is no built-in "isEmail" function, so we use a **regular expression (regex)** — a pattern that describes what a valid email looks like.

A valid email has three parts: **local part** + @ + **domain** with a dot and an extension.

SIMPLE AND PRACTICAL EMAIL REGEX

JAVASCRIPT — EMAIL VALIDATION FUNCTION

```
function isValidEmail(email) {
  // A widely used, reasonably strict pattern
  const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return pattern.test(email);
}

console.log(isValidEmail("riya@example.com"));     // true
console.log(isValidEmail("riya.example.com"));     // false (no @)
console.log(isValidEmail("riya@example"));         // false (no dot)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
console.log(isValidEmail("riya @example.com"));   // false (space)
console.log(isValidEmail(""));                    // false
```

BREAKING DOWN THE REGEX

Part	Meaning
`^`	Start of the string
`[^\s@]+`	One or more characters that are *not* a space or `@` (the local part)
`@`	A literal `@` symbol
`[^\s@]+`	One or more characters for the domain name
`\.`	A literal dot
`[^\s@]+`	One or more characters for the extension (com, in, org…)
`$`	End of the string

A STRICTER EMAIL REGEX

JAVASCRIPT — STRICTER PATTERN

```
function isValidEmailStrict(email) {
  const pattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/
  return pattern.test(email);
}

console.log(isValidEmailStrict("riya.sharma@example.co.in"));  // tr
console.log(isValidEmailStrict("riya@example.c"));             // fa
console.log(isValidEmailStrict("riya@@example.com"));          // fa
```

Part	Meaning
`[a-zA-Z0-9._%+-]+`	Allowed characters in the local part

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Part	Meaning
`[a-zA-Z0-9.-]+`	Domain name letters, digits, dots, hyphens
`\.[a-zA-Z]{2,}`	A dot followed by at least 2 letters

Best practice

Keep the regex simple and readable. Over-complicated email regexes reject valid addresses. The browser's `type="email"` plus a simple pattern is usually enough.

7.7 Complete working example — form + email validation

HTML — FULL PAGE

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    .field { margin-bottom: 14px; }
    .error { color: #ef4444; font-size: 0.85rem; }
    .success { color: #10b981; font-weight: bold; }
    input { padding: 8px; width: 260px; }
  </style>
</head>
<body>

  <h2>Contact Form</h2>

  <form id="contactForm" onsubmit="return validateContactForm(event)

    <div class="field">
      <label>Name:</label><br />
      <input type="text" id="cname" />
      <div class="error" id="cnameError"></div>
    </div>
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    <div class="field">
      <label>Email:</label><br />
      <input type="text" id="cemail" />
      <div class="error" id="cemailError"></div>
    </div>

    <div class="field">
      <label>Message:</label><br />
      <textarea id="cmsg" rows="4" cols="30"></textarea>
      <div class="error" id="cmsgError"></div>
    </div>

    <button type="submit">Send</button>
    <button type="reset">Clear</button>

    <div class="success" id="formSuccess"></div>

  </form>

  <script>
    function isValidEmail(email) {
      return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
    }

    function validateContactForm(e) {
      // reset messages
      document.getElementById("cnameError").textContent  = "";
      document.getElementById("cemailError").textContent = "";
      document.getElementById("cmsgError").textContent   = "";
      document.getElementById("formSuccess").textContent = "";

      const name  = document.getElementById("cname").value.trim();
      const email = document.getElementById("cemail").value.trim();
      const msg   = document.getElementById("cmsg").value.trim();

      let ok = true;

      if (name === "") {
        document.getElementById("cnameError").textContent = "Name is
        ok = false;
      } else if (name.length < 3) {
        document.getElementById("cnameError").textContent =
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            "Name must be at least 3 characters.";
          ok = false;
        }

        if (email === "") {
          document.getElementById("cemailError").textContent = "Email
          ok = false;
        } else if (!isValidEmail(email)) {
          document.getElementById("cemailError").textContent =
            "Enter a valid email like name@example.com";
          ok = false;
        }

        if (msg.length < 10) {
          document.getElementById("cmsgError").textContent =
            "Message must be at least 10 characters.";
          ok = false;
        }

        if (!ok) {
          e.preventDefault();
          return false;
        }

        document.getElementById("formSuccess").textContent =
          "Thank you, " + name + "! Your message has been validated.";
        e.preventDefault();       // demo only — do not actually submi
        return false;
      }
    </script>

</body>
</html>
```

How the flow works

1. User clicks Submit → the `submit` event fires.
2. Our function reads every field value.
3. Each rule is checked; failing rules write a message into the error `<div>`.
4. If anything failed, `preventDefault()` blocks submission.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5. If everything passed, the data can be sent to the server.

7.8 Real-time validation with input events

Instead of validating only on submit, you can validate as the user types.

JAVASCRIPT — LIVE VALIDATION

```
const emailInput = document.getElementById("cemail");
const emailError = document.getElementById("cemailError");

emailInput.addEventListener("input", function () {
  const value = this.value.trim();

  if (value === "") {
    emailError.textContent = "";
  } else if (!isValidEmail(value)) {
    emailError.textContent = "Invalid email format";
  } else {
    emailError.textContent = "";
    emailError.style.color = "#10b981";
    emailError.textContent = "✓ Looks good";
  }
});
```

7.9 Validation checklist

- Always `trim()` text values before checking them.
- Validate on **submit**, and optionally live on **input** / **blur**.
- Show *specific* messages — "Email is required" is better than "Invalid input".
- Reset all error messages at the start of every validation run.
- Do not rely only on client-side checks — validate again on the server.
- Use `type="email"` and `required` as a first line of defence.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

8 Quick Reference Summary

Topic	Key points to remember
Dialog boxes	`alert()` (message), `confirm()` (true/false), `prompt()` (text input). All are blocking.
Arrays	Ordered list, index starts at 0, `length` property. Methods: push, pop, shift, unshift, splice, slice, indexOf, includes, map, filter, reduce, sort, join.
Functions	Reusable blocks. Parameters in, `return` out. Declarations, expressions, arrow functions, callbacks.
String	`length, toUpperCase, toLowerCase, trim, indexOf, includes, slice, replace, split, padStart`
Math	`PI, abs, ceil, floor, round, max, min, pow, sqrt, random`
Date	`new Date()`, months are 0-based, getters like `getFullYear()`, formatters like `toLocaleDateString()`
Events	Attach with `addEventListener()`. Required set: onclick, ondblclick, onmouseover, onmouseout, onkeypress, onkeyup, onfocus, onblur, onload, onchange, onsubmit, onreset.
DOM	`getElementById, querySelector, querySelectorAll`; change with `innerHTML, textContent, value, style, classList`; build with `createElement + appendChild`.
History	`history.back(), forward(), go(n), length, pushState(), replaceState()`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Topic	Key points to remember
Validation	HTML5 attributes + JS onsubmit handler. Email regex: `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`

9 Practice Exercises

Write these in a single HTML file and test them in the browser. Use the Console (F12) to check results.

1. Ask the user's name with `prompt()` and greet them with `alert()`. If they cancel, show "No name entered".

2. Ask the user for two numbers and show their sum with `alert()`. Remember to convert the strings to numbers.

3. Use `confirm()` to ask "Delete this item?" and print a different message for OK and Cancel.

4. Create an array of 5 city names. Print the first, the last, and the total count.

5. Add a city to the start and another to the end of the array. Then remove the second element using `splice()`.

6. Given `[12, 45, 7, 30, 88, 21]`, use `filter()` to get numbers greater than 20, then `map()` to double them.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

7. Write a function `findLargest(arr)` that returns the largest number in an array, without using `Math.max`.

8. Write an arrow function `isEven(n)` that returns true or false. Use it to filter even numbers from an array.

9. Write a function `reverseString(str)` that reverses any text without using the built-in `reverse()` method.

10. Take a sentence and print: its length, its uppercase form, the number of words (using `split()`), and whether it contains the word "JavaScript".

11. Generate 5 random integers between 1 and 50 using `Math.random()` and store them in an array.

12. Use `Math` methods to round `4.567` up, down, and to the nearest integer. Print all three.

13. Create a Date object for today and print the year, month name, date, and day of the week.

14. Calculate how many days are left until 31 December of the current year.

15. Create a button and a paragraph. When the button is clicked, change the paragraph text and colour using the DOM.

16. Create an input box. Using `onkeyup`, display the number of characters typed so far in a paragraph.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

17. Build a form with Name and Email. Validate on submit and show error messages below each field. Use a regex for the email.

18. Add a list with 3 items. Using `createElement` and `appendChild`, add a 4th item from JavaScript when a button is clicked.

19. Add two buttons, "Back" and "Forward", that call `history.back()` and `history.forward()`.

20. Build a complete registration form with Full Name, Email, Password, Confirm Password, and Phone (10 digits). Validate everything with JavaScript and display all errors at once.

Final advice

Keep the Developer Console open while practising. Read every error message carefully — it usually tells you the exact line and the reason. Change values, break things on purpose, and fix them. That is how these concepts become permanent.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF